

Nama: Immanoel

NIM: 25151104

Kelas: 2B D3

GitHub: <https://github.com/tiranui/PBO-W4-042>



Bagian A

Circle.java

```
public class Circle {  
    private double radius;  
    private String color;  
    public Circle() {  
        radius = 1.0;  
        color = "red";  
    }  
    public Circle(double radius) {  
        this.radius = radius;  
        this.color = "red";  
    }  
    public Circle(double radius, String color) {  
        this.radius = radius;  
        this.color = color;  
    }  
    public double getRadius() {  
        return radius;  
    }  
    public double getArea() {  
        return radius * radius * Math.PI;  
    }  
    public void setRadius(double radius) {  
        this.radius = radius;  
    }  
    public String getColor() {  
        return color;  
    }  
}
```

```

}

public void setColor(String color) {
    this.color = color;
}

@Override
public String toString() {
    return "Circle[radius=" + radius + ", color=" + color + "]";
}
}

```

Cylinder.java

```

public class Cylinder extends Circle {
    private double height;

    public Cylinder() {
        super();
        height = 1.0;}

    public Cylinder(double height) {
        super();
        this.height = height;
    }

    public Cylinder(double radius, double height) {
        super(radius);
        this.height = height;
    }

    public double getHeight() {
        return height;
    }
}

```

```

public double getVolume() {
    return super.getArea() * height;
}

@Override
public double getArea() {
    return 2 * Math.PI * getRadius() * height
        + 2 * super.getArea();
}

@Override

public String toString() {

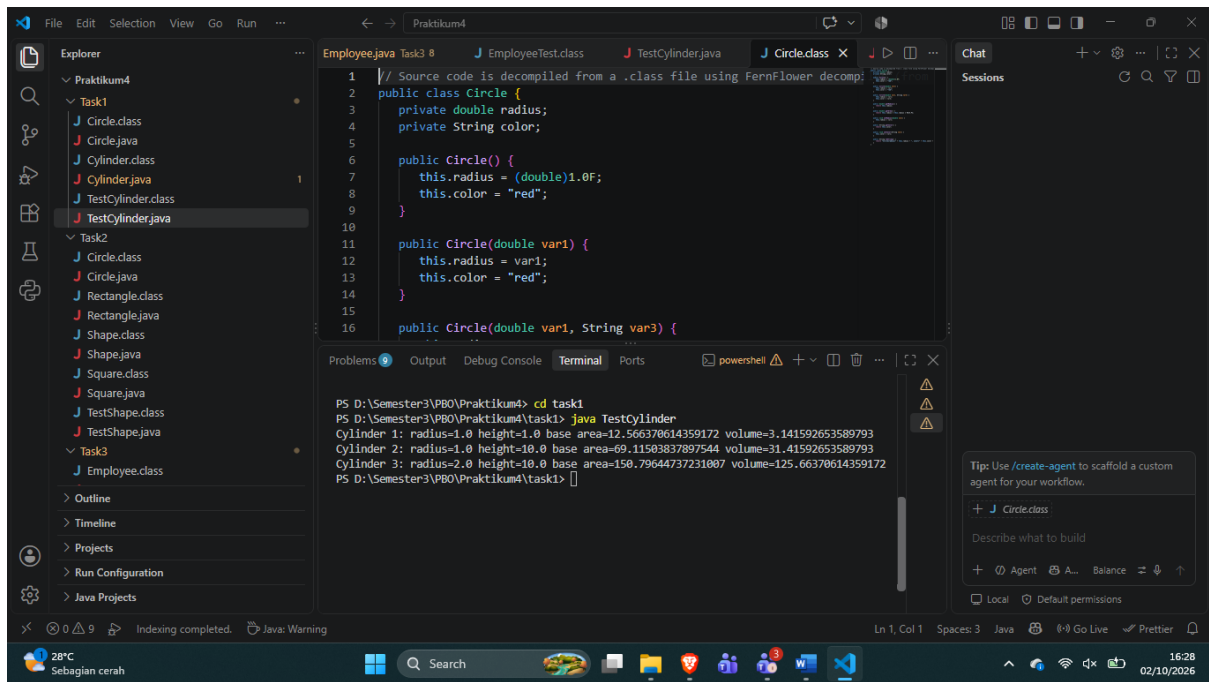
    return "Cylinder: subclass of " + super.toString()

        + " height=" + height;

}
}

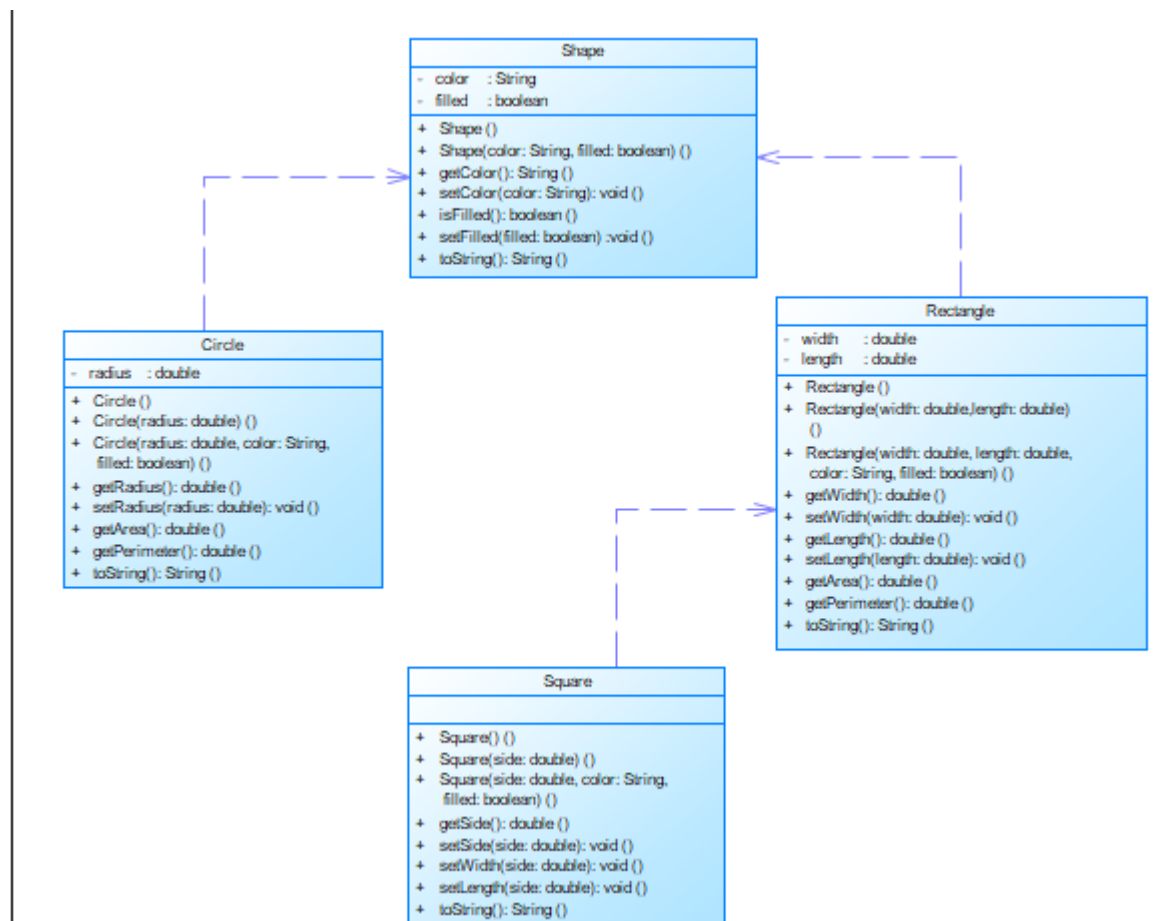
```

Output



Bagian B

Diagram



Source Code

Circle.java

```
public class Circle extends Shape {  
    private double radius;  
  
    public Circle() {  
        super();  
        radius = 1.0;  
    }  
  
    public Circle(double radius) {  
        super();  
        this.radius = radius;  
    }  
  
    public Circle(double radius, String color, boolean filled) {  
        super(color, filled);  
        this.radius = radius;  
    }  
  
    public double getRadius() { return radius; }  
  
    public void setRadius(double radius) { this.radius = radius; }  
  
    public double getArea() { return Math.PI * radius * radius; }  
  
    public double getPerimeter() { return 2 * Math.PI * radius; }  
  
    @Override  
    public String toString() {  
        return "Circle[radius=" + radius + ", " + super.toString() + "];"  
    }  
}
```

Rectangle.java

```
public class Rectangle extends Shape {  
    private double width;
```

```

private double length;

public Rectangle() {
    super();
    width = 1.0;
    length = 1.0;
}

public Rectangle(double width, double length) {
    super();
    this.width = width;
    this.length = length;
}

public Rectangle(double width, double length, String color, boolean filled) {
    super(color, filled);
    this.width = width;
    this.length = length;
}

public double getWidth() { return width; }

public void setWidth(double width) { this.width = width; }

public double getLength() { return length; }

public void setLength(double length) { this.length = length; }

public double getArea() { return width * length; } public double getPerimeter() { return 2 *
(width + length); }

@Override

public String toString() {
    return "Rectangle[width=" + width + ", length=" + length
+ ", " + super.toString() + "];"
}
}

```

Shape.java

// Source code is decompiled from a .class file using FernFlower decompiler (from IntelliJ IDEA).

```
public class Shape {  
    private String color;  
    private boolean filled;  
  
    public Shape() {  
        this.color = "green";  
        this.filled = true;  
    }  
  
    public Shape(String var1, boolean var2) {  
        this.color = var1;  
        this.filled = var2;  
    }  
  
    public String getColor() {  
        return this.color;  
    }  
  
    public void setColor(String var1) {  
        this.color = var1;  
    }  
  
    public boolean isFilled() {  
        return this.filled;  
    }  
}
```

```
}
```

```
public void setFilled(boolean var1) {
```

```
    this.filled = var1;
```

```
}
```

```
public String toString() {
```

```
    return "Shape[color=" + this.color + ", filled=" + this.filled + "];"
```

```
}
```

```
}
```

Square.java

```
public class Square extends Rectangle {
```

```
    public Square() {
```

```
        super(1.0, 1.0);
```

```
    }
```

```
    public Square(double side) {
```

```
        super(side, side);
```

```
    }
```

```
    public Square(double side, String color, boolean filled) {
```

```
        super(side, side, color, filled);
```

```
    }
```

```
    public double getSide() {
```

```
        return getWidth();
```

```
    }
```

```
    public void setSide(double side) {
```

```
        super.setWidth(side);
```

```
        super.setLength(side);
```

```

}

@Override

public void setWidth(double side) {
    super.setWidth(side);
    super.setLength(side);
}

@Override

public void setLength(double side) {
    super.setWidth(side);
    super.setLength(side);
}

@Override

public String toString() {
    return "Square[side=" + getSide() + ", " + super.toString() + "];"
}

}

```

TestShape.java

```

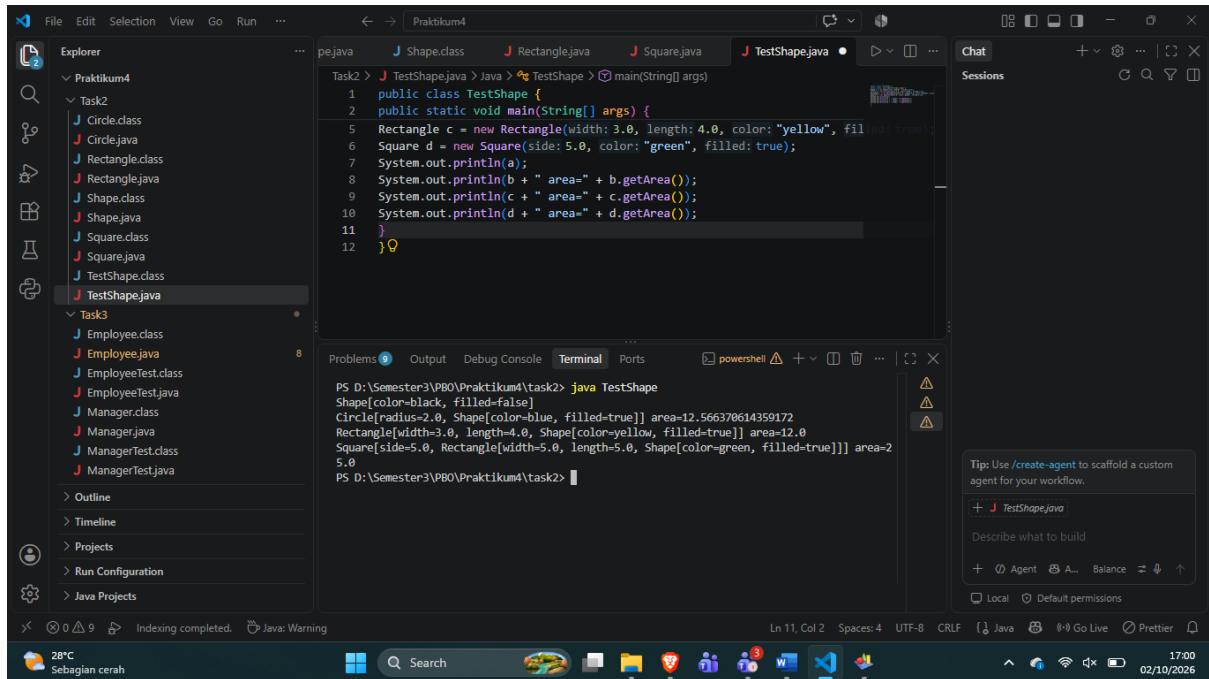
public class TestShape {

    public static void main(String[] args) {
        Shape a = new Shape("black", false);
        Circle b = new Circle(2.0, "blue", true);
        Rectangle c = new Rectangle(3.0, 4.0, "yellow", true);
        Square d = new Square(5.0, "green", true);
        System.out.println(a);
        System.out.println(b + " area=" + b.getArea());
        System.out.println(c + " area=" + c.getArea());
        System.out.println(d + " area=" + d.getArea());
    }
}

```

```
}  
  
}
```

Output:



The screenshot shows an IDE with a project named 'Praktikum4'. The Explorer panel on the left shows a file tree with 'Task2' containing 'Circle.class', 'Circle.java', 'Rectangle.class', 'Rectangle.java', 'Shape.class', 'Shape.java', 'Square.class', 'Square.java', 'TestShape.class', and 'TestShape.java'. The main editor shows 'TestShape.java' with the following code:

```
Task2 > J TestShape.java > Java > TestShape > main(String[] args)  
1 public class TestShape {  
2     public static void main(String[] args) {  
3  
4  
5         Rectangle c = new Rectangle(width: 3.0, length: 4.0, color: "yellow", filled: true);  
6         Square d = new Square(side: 5.0, color: "green", filled: true);  
7         System.out.println(a);  
8         System.out.println(b + " area=" + b.getArea());  
9         System.out.println(c + " area=" + c.getArea());  
10        System.out.println(d + " area=" + d.getArea());  
11    }  
12 }
```

The Output panel at the bottom shows the execution results:

```
PS D:\Semester3\VPB0\Praktikum4\task2> java TestShape  
Shape[color=black, filled=false]  
Circle[radius=2.0, Shape[color=blue, filled=true]] area=12.566370614359172  
Rectangle[width=3.0, length=4.0, Shape[color=yellow, filled=true]] area=12.0  
Square[side=5.0, Rectangle[width=5.0, length=5.0, Shape[color=green, filled=true]]] area=25.0  
PS D:\Semester3\VPB0\Praktikum4\task2>
```

Bagian C

Employee.java

```
public class Employee extends Sortable {  
  
    private String name;  
  
    private double salary;  
  
    private int hireday;  
  
    private int hiremonth;  
  
    private int hireyear;  
  
    public Employee(String n, double s, int day, int month, int year) {  
  
        name = n;  
  
        salary = s;  
  
        hireday = day;
```

```

        hiremonth = month;

        hireyear = year;
    }

    public void print() {

        System.out.println(name + " " + salary + " " + hireYear());
    }

    public void raiseSalary(double byPercent) {

        salary *= 1 + byPercent / 100;
    }

    public int hireYear() {

        return hireyear;
    }

    public String getName() {

        return name;
    }

    public double getSalary() {

        return salary;
    }

    @Override

    public int compare(Sortable b) {

        Employee eb = (Employee) b;

        if (salary < eb.salary) {

            return -1;
        }

        if (salary > eb.salary) {

            return 1;
        }

        return 0;
    }

```

```
}  
}
```

Manager.java

```
public class Manager extends Employee {  
    private String secretaryName;  
    public Manager(String n, double s, int d, int m, int y) {  
        super(n, s, d, m, y);  
        secretaryName = "";  
    }  
    @Override  
    public void raiseSalary(double byPercent) {  
        java.util.GregorianCalendar todaysDate =  
            new java.util.GregorianCalendar();  
        int currentYear =  
            todaysDate.get(java.util.Calendar.YEAR);  
        double bonus = 0.5 * (currentYear - hireYear());  
        super.raiseSalary(byPercent + bonus);  
    }  
    public String getSecretaryName() {  
        return secretaryName;  
    }  
    public void setSecretaryName(String secretaryName) {  
        this.secretaryName = secretaryName;  
    }  
}
```

Sortable.java

```

public abstract class Sortable {

    public abstract int compare(Sortable b);

    public static void shell_sort(Sortable[] a) {

        int n = a.length;

        // Shell sort menggunakan gap yang semakin kecil.

        for (int gap = n / 2; gap > 0; gap /= 2) {

            for (int i = gap; i < n; i++) {

                Sortable temp = a[i];

                int j = i;

                while (j >= gap && a[j - gap].compare(temp) > 0) {

                    a[j] = a[j - gap];

                    j -= gap;

                }

                a[j] = temp;

            }

        }

    }

}

```

EmployeeTest.java

```

public class EmployeeTest {

    public static void main(String[] args) {

        Employee[] staff = new Employee[3];

        staff[0] = new Employee(

```

```
"Antonio Rossi", 2000000, 1, 10, 1989);
```

```
staff[1] = new Manager(
```

```
"Maria Bianchi", 2500000, 1, 12, 1991);
```

```
staff[2] = new Employee(
```

```
"Isabel Vidal", 3000000, 1, 11, 1993);
```

```
for (int i = 0; i < staff.length; i++) {
```

```
    staff[i].raiseSalary(5);
```

```
}
```

```
System.out.println("Sebelum sorting:");
```

```
for (int i = 0; i < staff.length; i++) {
```

```
    staff[i].print();
```

```
}
```

```
Sortable.shell_sort(staff);
```

```
System.out.println("\nSetelah sorting berdasarkan salary:");
```

```
for (int i = 0; i < staff.length; i++) {  
  
    staff[i].print();  
  
}  
  
System.out.println("\nHasil compare:");  
  
System.out.println("staff[0] compare staff[1] = "  
  
    + staff[0].compare(staff[1]));  
  
System.out.println("staff[1] compare staff[2] = "  
  
    + staff[1].compare(staff[2]));  
  
}  
  
}
```

ManagerTest.java

```
public class ManagerTest{  
    public static void main (String[] args){  
        Employee[] staff = new Employee[3];  
        staff[0] = new Employee("Antonio Rossi", 2000000, 1, 10, 1989);
```

```

staff[1] = new Manager("Maria Bianchi", 2500000, 1, 12, 1991);

staff[2] = new Employee("Isabel Vidal", 3000000, 1, 11, 1993);

int i;

for (i = 0; i < 3; i++) staff[i].raiseSalary(5);

for (i = 0; i < 3; i++) staff[i].print();

}

}

```

Output:

```

Task3 > J ManagerTest.java > Language Support for Java(TM) by Red Hat > ManagerTest > main(String[])
1
2 public class ManagerTest{
3     public static void main (String[] args){
4         Employee[] staff = new Employee[3];
5         staff[0] = new Employee(n: "Antonio Rossi", s: 2000000, day: 1, month: 10, year: 1989);
6         staff[1] = new Manager(n: "Maria Bianchi", s: 2500000, d: 1, m: 12, y: 1991);
7         staff[2] = new Employee(n: "Isabel Vidal", s: 3000000, day: 1, month: 11, year: 1993);
8         int i;
9         for (i = 0; i < 3; i++) staff[i].raiseSalary(5);
10
11     }
12 }

```

Problems 12 Output Debug Console Terminal Ports powershell

Sebelum sorting:
Antonio Rossi 2100000.0 1989
Maria Bianchi 3062500.0 1991
Isabel Vidal 3150000.0 1993

Setelah sorting berdasarkan salary:
Antonio Rossi 2100000.0 1989
Maria Bianchi 3062500.0 1991
Isabel Vidal 3150000.0 1993

Hasil compare:
staff[0] compare staff[1] = -1
staff[1] compare staff[2] = -1
PS D:\Semester3\PS0\Praktikum4\Task3>

ID	Pass/Fail
T4-01	Pass
T4-02	Pass
T4-03	Pass
T4-04	Pass
T4-05	Pass
T4-06	Pass
T4-07	Pass

Bagian 7.3

1. Apa yang membuat Circle layak menjadi class yang lebih umum daripada Cylinder?
Circle lebih umum karena Circle hanya memiliki konsep dasar berupa radius, warna, dan luas lingkaran. Sedangkan Cylinder membutuhkan konsep tersebut sebagai

bagian dari alasnya dan menambahkan height serta perhitungan volume. Jadi Cylinder dapat dibuat sebagai turunan dari Circle karena Cylinder masih menggunakan konsep dasar yang dimiliki Circle.

2. Berdasarkan Step 5, apa arti praktis extends bagi object Cylinder?
extends Circle berarti class Cylinder merupakan subclass dari Circle. Secara praktis, object Cylinder dapat menggunakan method yang diwarisi dari Circle, seperti getRadius(), getColor(), dan method lainnya yang dapat diwariskan. Cylinder juga dapat menambahkan atribut dan method sendiri, seperti height dan getVolume().
3. Mengapa constructor Circle dijalankan ketika Anda membuat object Cylinder?
Karena Cylinder merupakan turunan dari Circle. Ketika object Cylinder dibuat, bagian object yang berasal dari Circle juga harus diinisialisasi. Hal tersebut dilakukan melalui super() atau super(radius) pada constructor Cylinder
4. Mengapa getVolume() menjadi salah setelah getArea() dioverride?
Awalnya getArea() pada Circle digunakan untuk menghitung luas lingkaran, sehingga dapat digunakan sebagai luas alas Cylinder.
Setelah getArea() dioverride di Cylinder, method tersebut berubah menjadi perhitungan luas permukaan Cylinder
5. Apa yang secara eksplisit Anda minta ketika menulis super.getArea() atau super.toString()?

Saya secara eksplisit meminta method milik superclass untuk dijalankan.

- super.getArea() → menjalankan getArea() milik Circle.
- super.toString() → menjalankan toString() milik superclass.

Jadi kita tidak menggunakan method yang sedang dioverride di subclass.

6. Mengapa Square perlu mengubah setter yang diwarisi dari Rectangle?

Karena Rectangle memiliki width dan length yang dapat berbeda. Sedangkan pada Square, kedua sisi harus selalu sama.

7. Apakah setiap dua class yang mirip harus menggunakan inheritance? Jelaskan dari salah satu kasus Anda.

Tidak. Dua class yang memiliki beberapa kesamaan belum tentu harus menggunakan inheritance. Yang lebih penting adalah apakah terdapat hubungan class umum dan class yang lebih khusus.

Pada kasus Employee dan Manager, inheritance sesuai karena Manager merupakan jenis Employee. Manager membutuhkan atribut dan method dasar Employee, tetapi memiliki perilaku tambahan pada raiseSalary().

8. Lengkapi dengan bahasa Anda sendiri: "Inheritance bukan sekadar mengurangi duplikasi, tetapi ..."

inheritance bukan sekadar mengurangi duplikasi, tetapi juga membentuk hubungan antara class yang lebih umum dengan class yang lebih khusus. Dengan inheritance, subclass dapat menggunakan kemampuan dari superclass, menambahkan kemampuan baru, dan mengubah perilaku tertentu melalui overriding sesuai kebutuhannya.